

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 22
Graph (III)

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

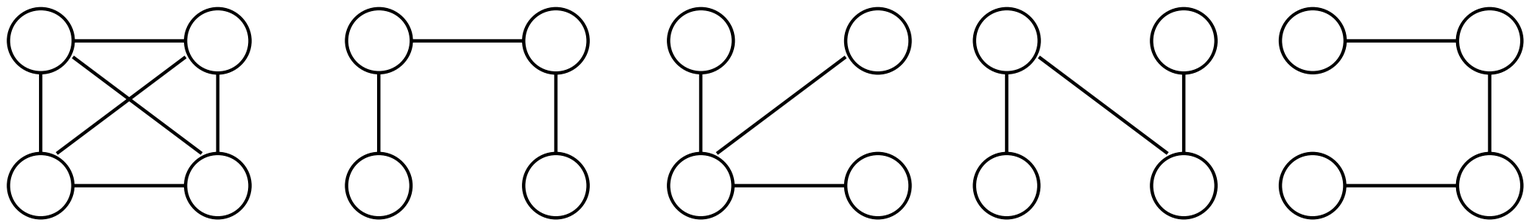
Outline

- Minimum Spanning Trees
  - Prim's Algorithm
  - Kruskal's Algorithm

最小生成树

Minimum Spanning Trees

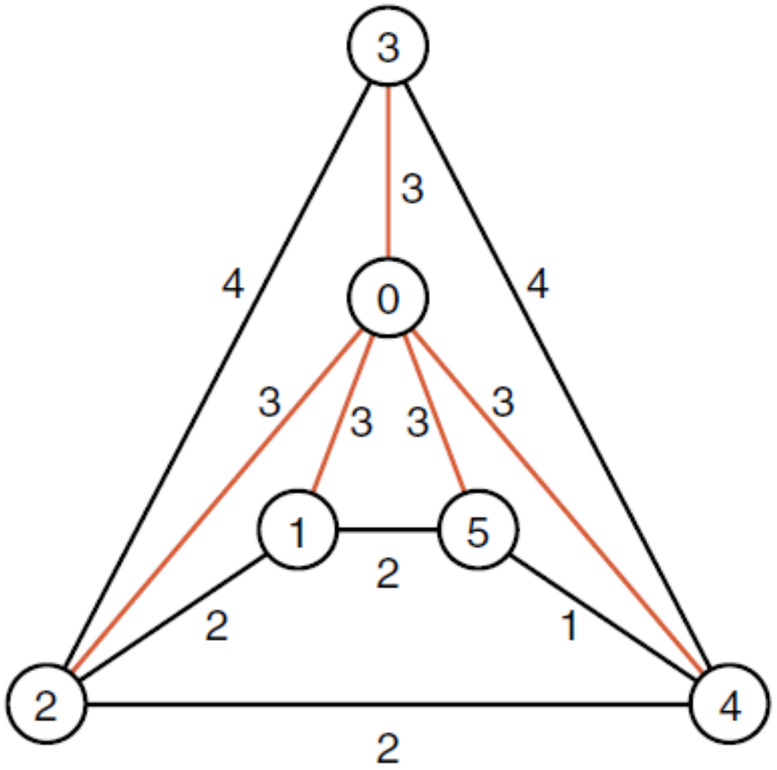
- A tree that is build up out of all the vertices and some of the edges of *G* is called a spanning tree of *G.*
  - Undirected
  - Connected
  - Acyclic
- A minimum spanning tree (MST) of a connected network is a spanning tree such that the sum of the weights of its edges is as small as possible.



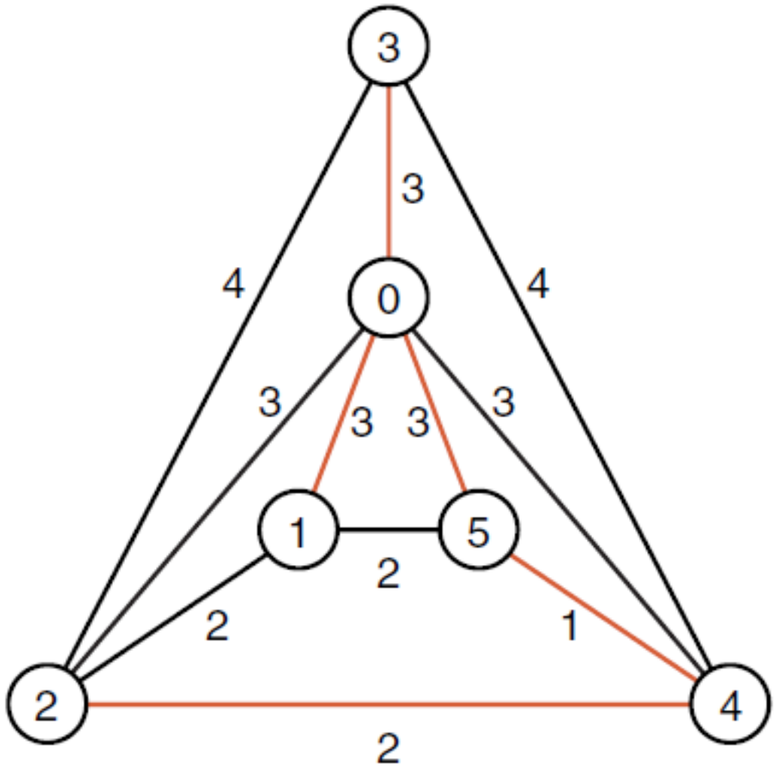
A connected, undirected graph

Four of the spanning trees of the graph

Two Spanning Trees



Weight sum of tree = 15
(a)



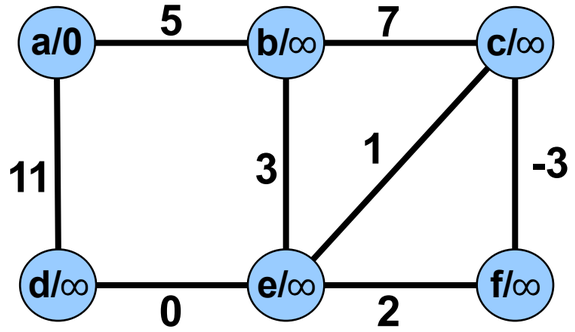
Weight sum of tree = 12
(b)

Prim's Algorithm for Minimum Spanning Trees

- Prim's algorithm for finding an MST is a greedy algorithm.
- Start by selecting an arbitrary vertex, include it into the current MST.
- Grow the current MST by inserting into it the vertex closest to one of the vertices already in the current MST.
- Repeat until all vertices are in the tree.

选择边的顶点
(不能形成回路)

Example of Prim's Algorithm

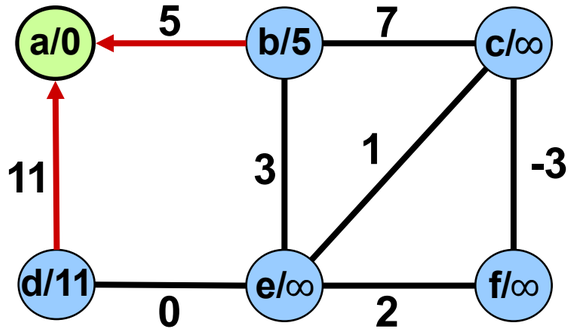


Not in tree

Q = a b c d e f

0 ∞ ⟷ ∞

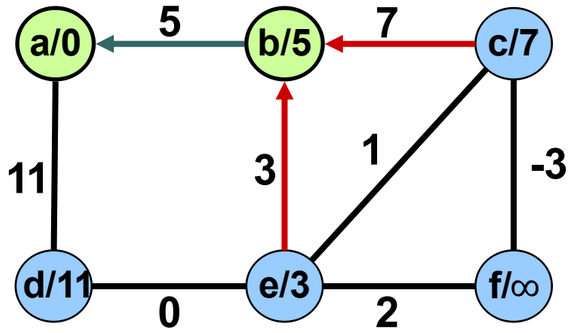
Example of Prim's Algorithm



Q = b d c e f

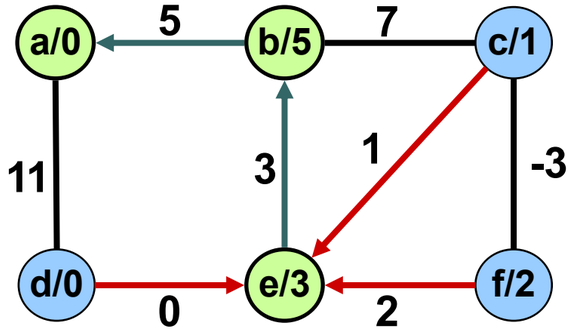
5 11 ∞ ∞

Example of Prim's Algorithm



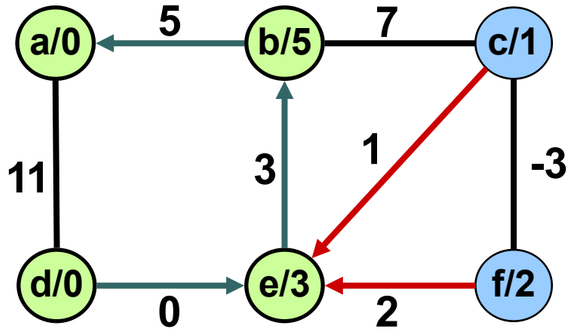
Q = e c d f
3 7 11 ∞

Example of Prim's Algorithm



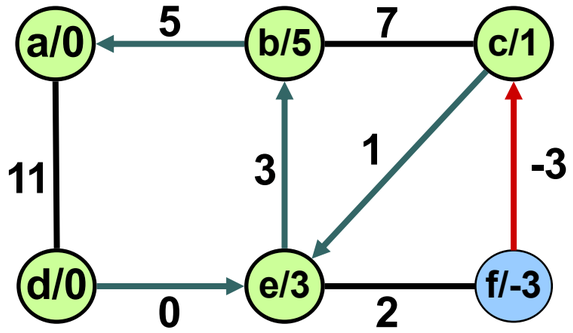
Q = d c f
0 1 2

Example of Prim's Algorithm



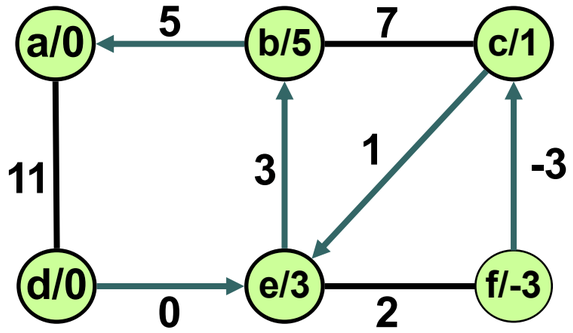
Q = c f
1 2

Example of Prim's Algorithm



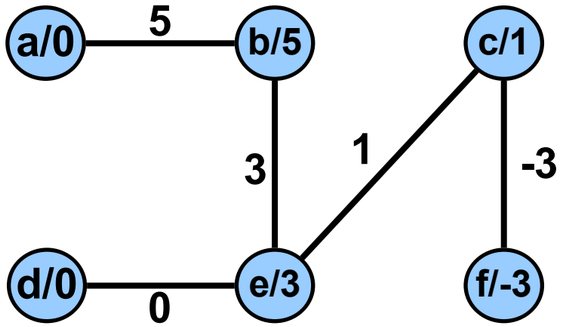
Q = f
-3

Example of Prim's Algorithm



Q = ∅

Example of Prim's Algorithm



Implementation of Prim's Algorithm

```
template <class Weight, int graph_size>
void Network < Weight, graph_size > ::minimal_spanning(Vertex source,
                                Network<Weight, graph_size> &tree) const
/* Post: The Network tree contains a minimal spanning tree for the connected com-
         ponent of the original Network that contains vertex source. */
{ tree.make_empty(count);
  bool component[graph_size];   //   Vertices in set X
  Weight distance[graph_size];  //   Distances of vertices adjacent to X
  Vertex neighbor[graph_size];  //   Nearest neighbor in set X
  Vertex w;
  for (w = 0; w < count; w++) {
    component[w] = false;
    distance[w] = adjacency[source][w];
    neighbor[w] = source;      }
```

If there is no edge between (u,v), then adjacency[u][v]=+∞

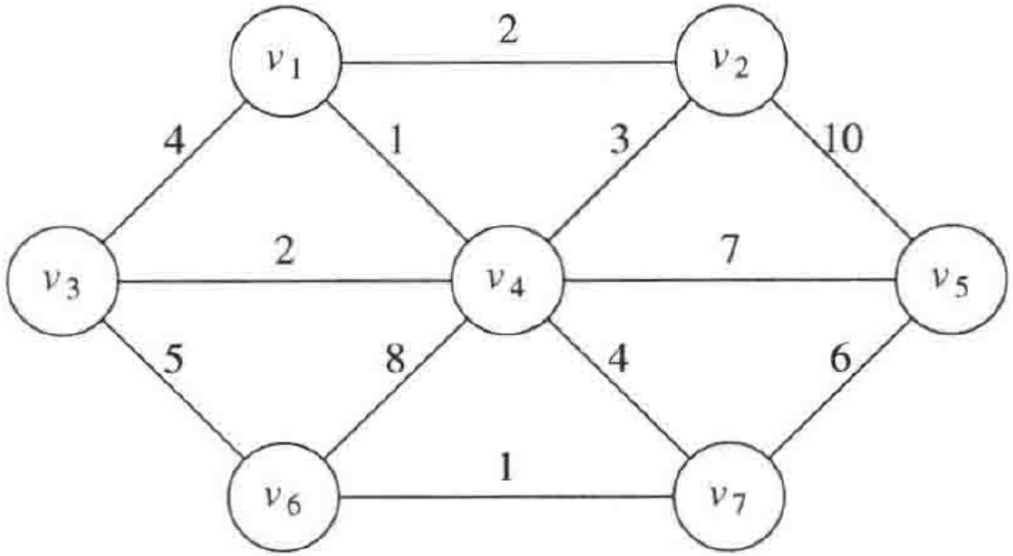
```
component[source] = true;   //   source alone is in the set X.
for (int i = 1; i < count; i++) {
   Vertex v;                //   Add one vertex v to X on each pass.
   Weight min = infinity;
   for (w = 0; w < count; w++) if (!component[w])
      if (distance[w] < min) {
         v = w;
         min = distance[w]; }
   if (min < infinity) {
      component[v] = true;
      tree.add_edge(v, neighbor[v], distance[v]);
      for (w = 0; w < count; w++) if (!component[w])
         if (adjacency[v][w] < distance[w]) {
            distance[w] = adjacency[v][w];
            neighbor[w] = v;                    }
   }
   else break;              //   finished a component in disconnected graph
  }
}
```

Find the vertex in Q closest to the current MST

Update the distance

Exercise

- Use Prim's algorithm to construct a minimum spanning tree of the following graph, rooted at v_5.



Time Complexity

A Generic MST Algorithm:

```
Q := V[G];
for each u ∈ Q do
      dis[u] := ∞
od;
dis[r] := 0;
π[r] := NIL;
while Q ≠ ∅ do
      u := Extract - Min(Q);
      for each v ∈ Adj[u] do
            if v ∈ Q and w(u, v) < dis[v]
                  then  π[v] := u;
                        dis[v] := w(u, v)
            fi
      od
od
```

Complexity:

Naive implementation:

$O(V^2)$

Using binary heaps:

$O((V+E) \lg V)$.

Using Fibonacci heaps:

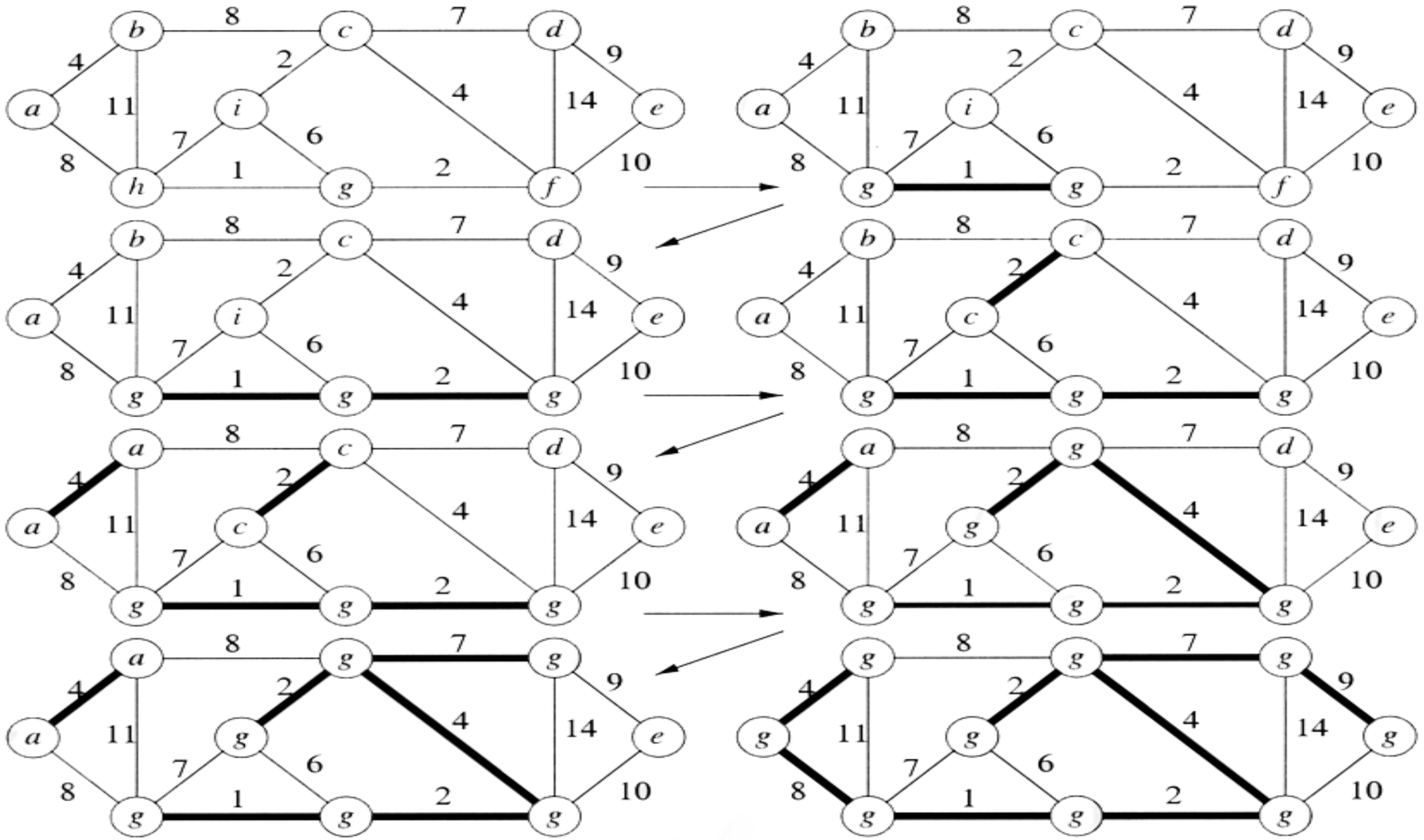
$O(E + V \lg V)$.

Kruskal's Algorithm for Minimum Spanning Trees

把所有顶点当成树(孤立的结点)

- Kruskal's Algorithm is directly based on the generic MST algorithm. It builds the MST in forest.
- Initially, each vertex is in its own tree in forest. Then, the algorithm considers each edge in turn, order by increasing weight.
- If an edge (*u*, *v*) connects two different trees, then (*u*, *v*) is added to the set of edges of the MST, and two trees connected by an edge (*u*, *v*) are merged into a single tree.
- On the other hand, if an edge (*u*, *v*) connects two vertices in the same tree, then edge (*u*, *v*) is discarded.

Kruskal's Algorithm for Minimum Spanning Trees



Kruskal's Algorithm

```
1  A ← ∅                                  // initially A is empty
2  for each vertex v ∈ V[G]               // line 2-3 takes O(V) time
3       do Make-Set(v)                    // create set for each vertex
4  sort the edges of E into non-decreasing order by weight w
5  for each edge (u,v) ∈ E, taken in non-decreasing order by weight  do
6       if Find-Set(u) ≠ Find-Set(v)          // u&v on different trees
7            then  A ← A ∪ {(u,v)}
8                  Union(u,v)
9  return A
```

- **Make_Set**(*v*): Create a new set whose only member is pointed to by *v*. Note that for this operation *v* must already be in a set.
- **Find_Set**(*v*): Returns a pointer to the set containing *v*.
- **Union**(*u*, *v*): Unites the dynamic sets that contain *u* and *v* into a new set that is union of these two sets.

Implementation of Kruskal's Algorithm

```
int p[maxn],r[maxm];

int cmp(int i, int j) {return w[i]<w[j]};

int find(int x)
{return (p[x] == -1) ? x : p[x]=find(p[x]);}

int union(int x, int y) { p[y] = x;}

int Kruskal() {
  int ans = 0;
  for(int i=0;i<n;i++) p[i] = -1;
  for(int i=0;i<m;i++) r[i] = i;
  sort(r,r+m,cmp);
  for(int i=0;i<m;i++) {
    int e = r[i], x = find(u[e]), y = find(v[e]);
    if (x!=y) {ans += w[e]; union(x,y);}
  }
  return ans;
}
```

Time Complexity

- MAKE-SETs requires O(V)
- Edge sorting requires O(E logE)
- FIND-SETs and UNIONs perform O(E) time. Assuming the implementation of disjoint-set data structure that uses union by rank and path compression, so amortized in O(E α(V)). (α为反ackerman函数)
- α(V) = O(log V) = O(log E)
- Total Time: O(E logE) = O(E logV)
- If edges are already sorted, O(E α(V)), which is almost linear.

Templates

```cpp
struct edge{int v,w;};
vector<edge> e[N];
int d[N], vis[N];

bool prim(int s){
  for(int i=0;i<=n;i++)d[i]=inf;
  d[s]=0;
  for(int i=1;i<=n;i++){
    int u=0;
    for(int j=1;j<=n;j++)
      if(!vis[j]&&d[j]<d[u]) u=j;
    vis[u]=1; //标记u已出圈
    ans+=d[u];
    if(d[u]!=inf) cnt++;
    for(auto ed : e[u]){
      int v=ed.v, w=ed.w;
      if(d[v]>w) d[v]=w;
    }
  }
  return cnt==n;
}
```

$O(n^2)$

```cpp
int n,m,s,a,b,c,ans,cnt;
struct edge{int v,w;};
vector<edge> e[N];
int d[N], vis[N];
priority_queue<pair<int,int>> q;

bool prim(int s){
  for(int i=0;i<=n;i++) d[i]=inf;
  d[s]=0; q.push({0,s});
  while(q.size()){
    int u=q.top().second; q.pop();
    if(vis[u])continue;//再出队跳过
    vis[u]=1;//标记u已出队
    ans+=d[u]; cnt++;
    for(auto ed : e[u]){
      int v=ed.v, w=ed.w;
      if(d[v]>w){
        d[v]=w;
        q.push({-d[v],v});//大根堆
      }
    }
  }
  return cnt==n;
}
```

$O(mlogm)$

Lazy implementation of Prim

```cpp
struct edge{
  int u,v,w;
  bool operator<(const edge &t)const
  {return w < t.w;}
}e[N];
int fa[N],ans,cnt;

int find(int x){
  if(fa[x]==x) return x;
  return fa[x]=find(fa[x]);
}
bool kruskal(){
  sort(e,e+m);
  for(int i=1;i<=n;i++)fa[i]=i;
  for(int i=0; i<m; i++){
    int x=find(e[i].u);
    int y=find(e[i].v);
    if(x!=y){
      fa[x]=y;
      ans+=e[i].w;
      cnt++;
    }
  }
  return cnt==n-1;
}
```

$O(mlogm)$

References

- https://www.bilibili.com/video/BV1f54y1d7ba/
  - Extended learning: Try to understand lazy implementation and eager implementation of Prim's algorithm
- https://www.bilibili.com/video/BV1gY4y1B7vd/

Thank you!

